DESENVOLVIMENTO MOBILE

ALEX ZACHARIAS

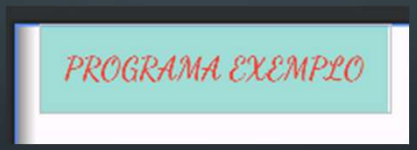
ALEX@EINSTEINLIMEIRA.COM.BR

ANDROID - TEXTVIEW

- Algumas propriedades do TextView:
- text: Texto que será exibido
- background / textColor: Cor do fundo ou da fonte de letra
- layout_width / layout_height: largura e altura do componente
- layout_margin: margem do componente em relação ao componente pai
- padding: margem interna do componente
- textSize: tamanho da fonte de letra, devemos utilizar a medida sp
- textStyle: estilo do texto, como itálico ou negrito por exemplo
- textAllCaps: se verdadeiro coloca todas as letras em maiúsculo
- fontFamily: qual fonte desejamos utilizar, arial por exemplo.

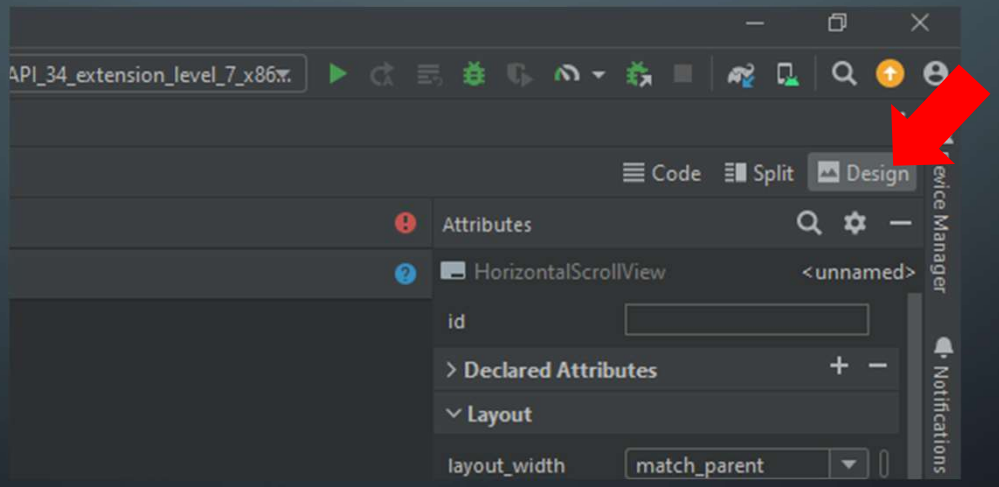
ANDROID - TEXTVIEW

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="16dp"
    android:background="@color/fundo_texto"
    android:padding="16dp"
    android:text="Programa exemplo"
    android:textColor="@color/texto_ver"
    android:textSize="20sp"
    android:textAllCaps="true"
    android:fontFamily="cursive"
    android:textStyle="bold"/>
```



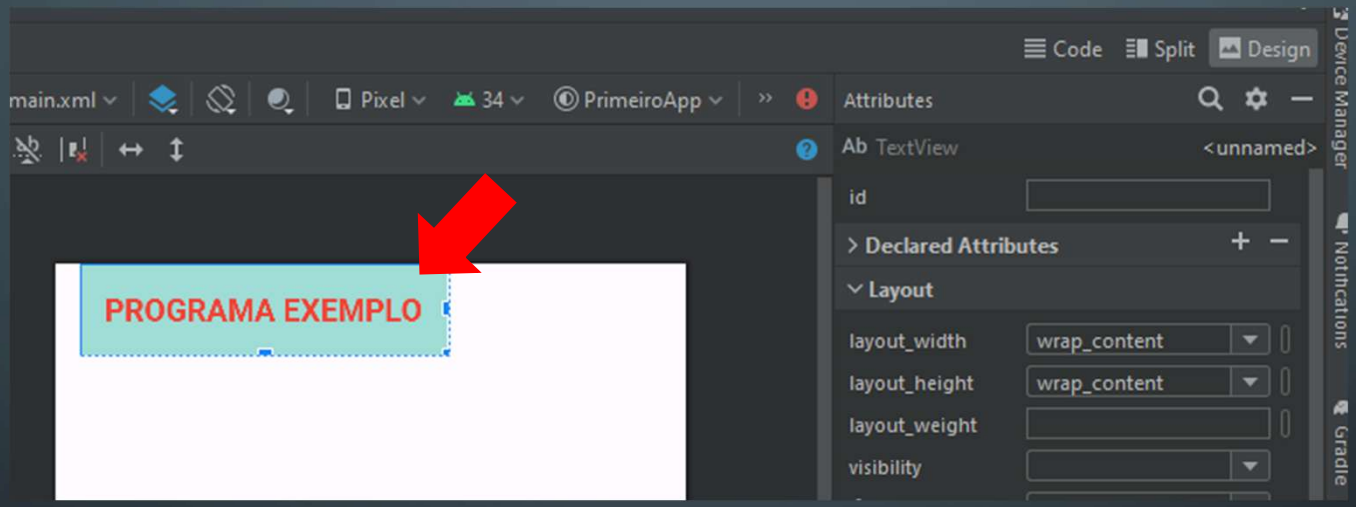
ANDROID - TEXTVIEW

- Por padrão o Android tem poucas fontes de letras instaladas, mas podemos incluir mais fontes, para isso, mude a sua visualização para a opção Design:



ANDROID - TEXTVIEW

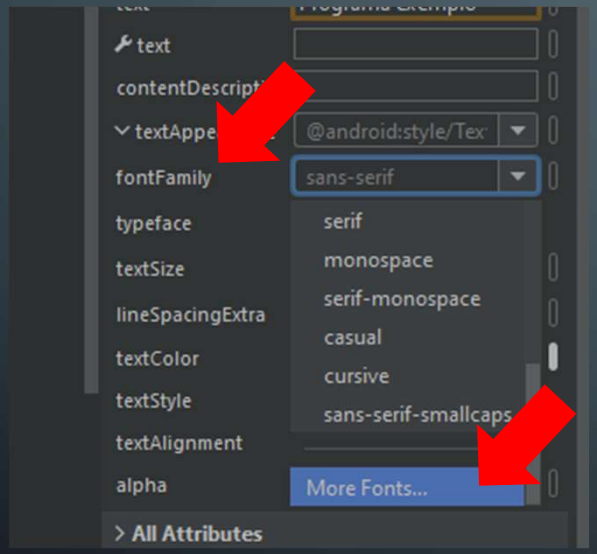
- Agora clique no elemento:



- Note que no lado direito da tela temos um caixa com as propriedades do elemento, podemos fazer as alterações por ali.

ANDROID - TEXTVIEW

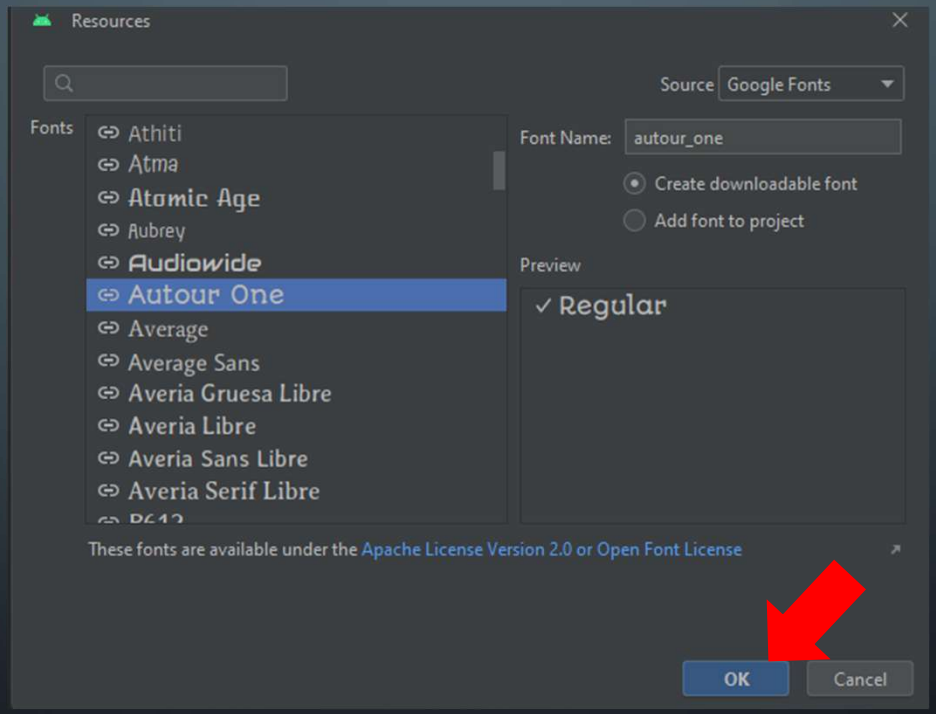
- Procure nessa caixa a opção fontFamily e clique nela, role o menu que vai abrir ate o final e clique em More Fonts...



Não conseguimos acessar essa opção de mais fontes diretamente pelo código, para isso temos que acessar esse menu.

ANDROID - TEXTVIEW

- Escolha uma fonte na janela vai abrir e clique em OK, pronto essa fonte foi adicionada ao seu aplicativo.



ANDROID – IDENTIFICAÇÃO DE ELEMENTOS

- Elementos que vamos interagir pelo nosso código Kotlin precisam de uma identificação para que possamos se referir a eles.
- Imagine que de dentro do código vamos alterar o texto que é exibido em um TextView, como saberemos para qual TextView enviar o texto?
- Para isso todos os elementos visuais possuem um atributo id, no qual podemos colocar um nome para podermos acessar o elemento. Seu conteúdo segue um padrão devendo ser sempre iniciado com @+id/

ANDROID – IDENTIFICAÇÃO DE ELEMENTOS

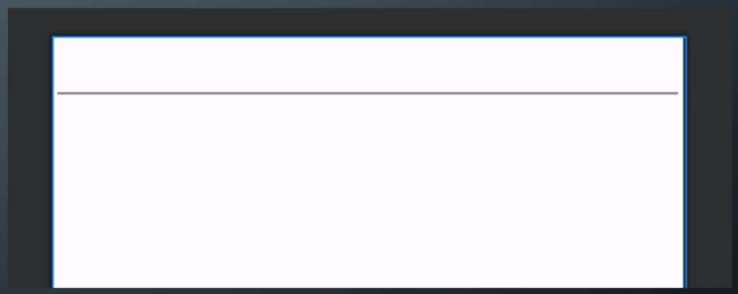
```
<TextView
    android:id="@+id/text_exemplo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Exemplo de Texto"
    android:textSize="16sp"/>
```

- Criamos um TextView com o id text_exemplo.
- Para padronizar devemos começar o id do componente com o nome do mesmo, no nosso caso text por ser um textView.

ANDROID - EDITTEXT

- EditText permite que o usuário digite um texto para enviar dados para o aplicativo.
- Ele possui algumas propriedades igual ao TextView como posição na tela, cores, fonte de letra entre outros.

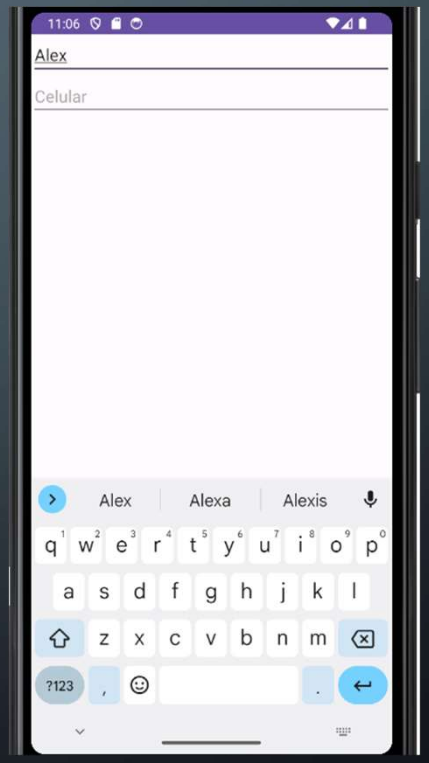
```
<EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```



ANDROID - EDITTEXT

- O atributo hint escreve o texto dentro do EditText para o usuário saber a que esse campo se refere, ao digitar um valor no campo o hint desparece.

```
<EditText
    android:id="@+id/edit_nome"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Nome" />

<EditText
    android:id="@+id/edit_celular"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Celular" />
```



ANDROID - EDITTEXT

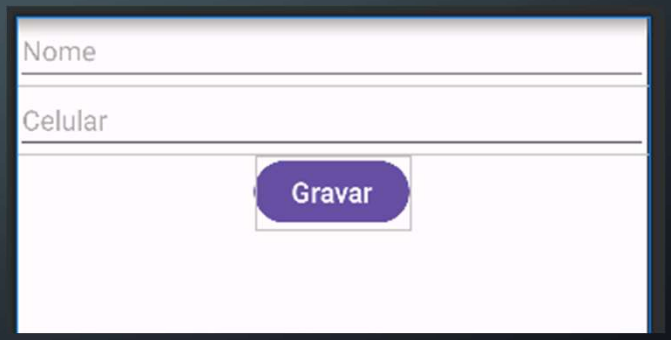
- O atributo inputType permite filtrar o tipo de dado que vai ser digitado nesse campo, como por exemplo números, data, email, senhas...

```
<EditText
    android:id="@+id/edit_celular"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="phone"
    android:hint="Celular" />
```

ANDROID - BUTTON

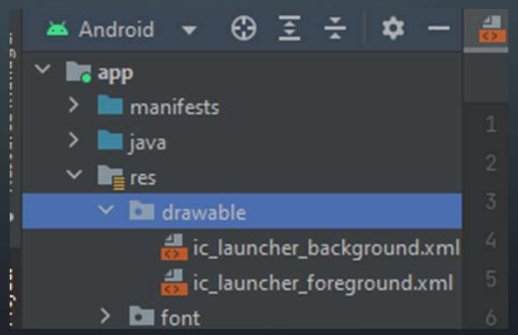
- Button é o botão onde o usuário ira clicar para executar alguma função.
- Ele possui os atributos básicos para posicionamento e definição de texto igual aos outros componentes:

```
<Button
    android:id="@+id/button_grava"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:text="Gravar"
    android:textSize="18sp" />
```



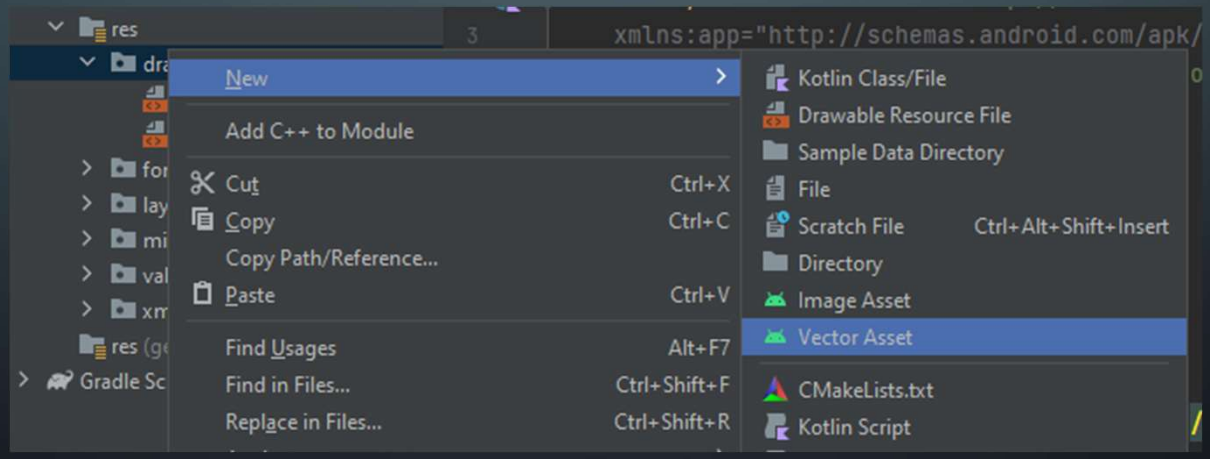
ANDROID - IMAGEVIEW

- Para exibirmos imagens na tela utilizamos o componente ImageView.
- Ele também utiliza os atributos de localização na tela e tamanho.
- Seu atributo principal é o src que indica qual imagem vamos exibir
- Toda imagem deve ser adicionada na pasta Drawable dentro da pasta res.



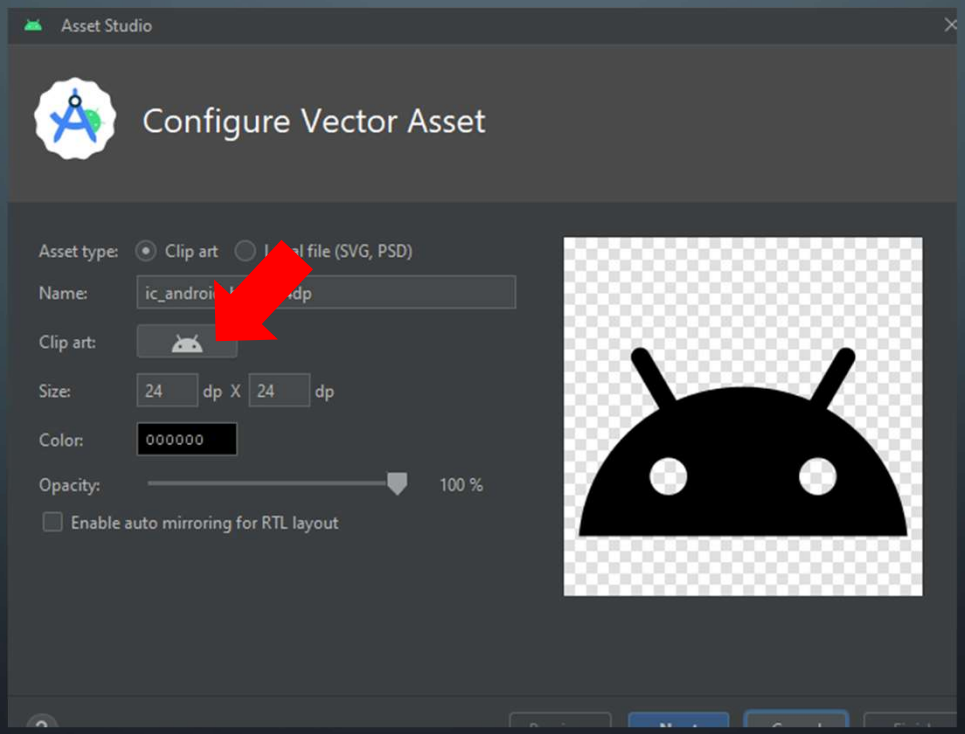
ANDROID - IMAGEVIEW

- Por enquanto vamos trabalhar com vetores, que podem ser exibidos de qualquer tamanho sem perder a resolução.
- Para incluir um vetor clique com o botão direito sobre a pasta Drawable, vá em New – Vector Asset



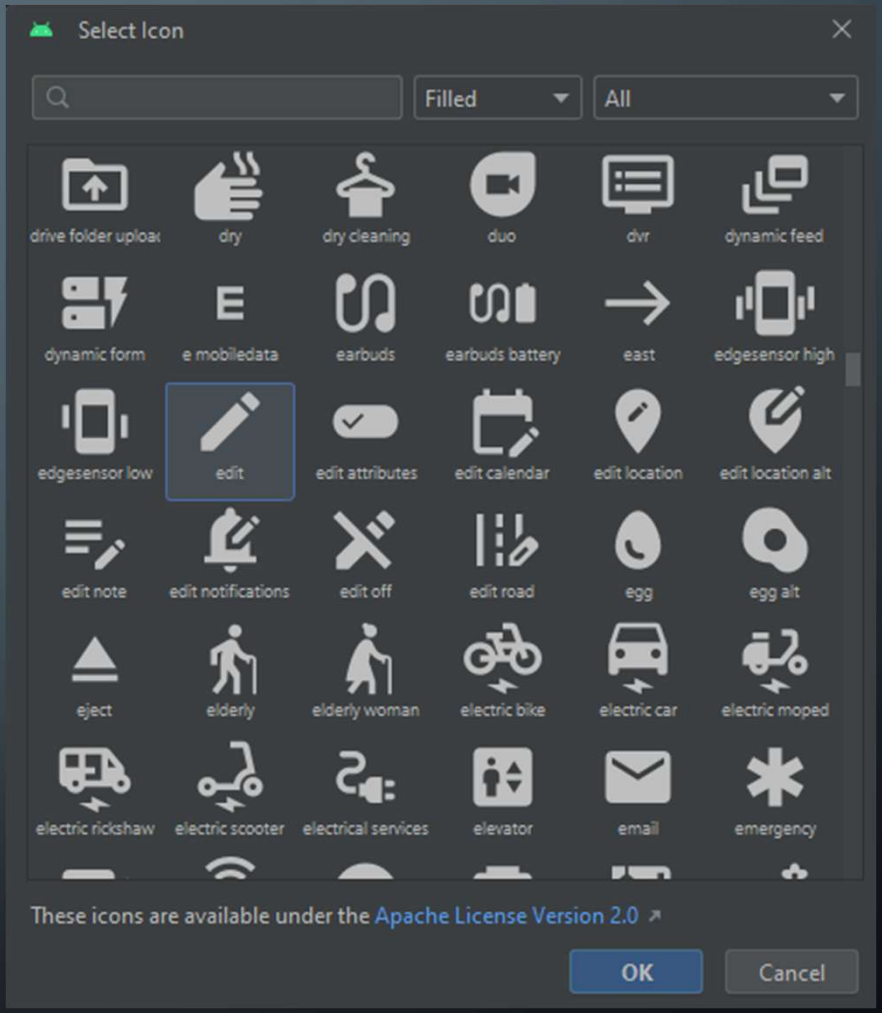
ANDROID - IMAGEVIEW

- Na janela que abriu, clique no desenho que esta a direita de Clip Art para selecionar sua imagem:



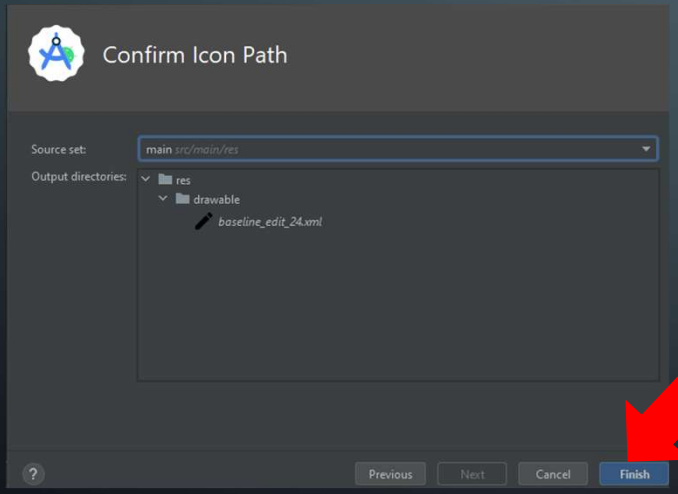
ANDROID - IMAGEVIEW

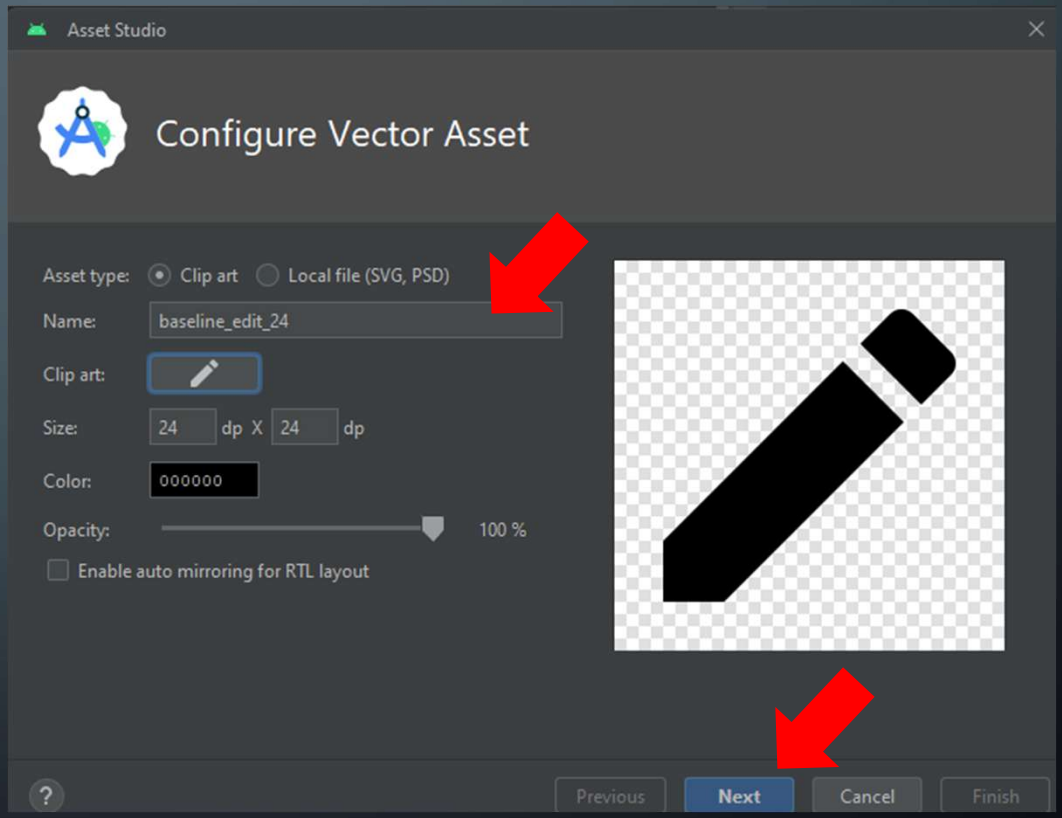
- Procure a imagem desejada e clique em OK:



ANDROID - IMAGEVIEW

- Se desejar altere o nome da imagem, as outras opções podem ser alteradas dentro do .xml
- Clique em Next e Finish na próxima tela.





ANDROID - IMAGEVIEW

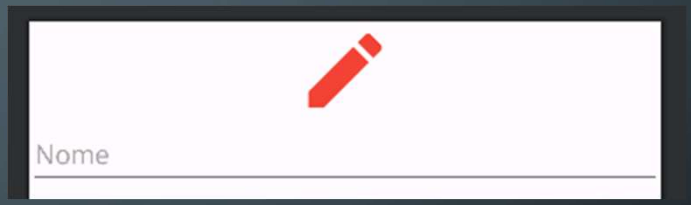
- Agora que a imagem esta dentro do projeto, podemos utilizar ela com o atributo src:

```
<ImageView
    android:src="@drawable/baseline_edit_24"
    android:layout_gravity="center_horizontal"
    app:tint="@color/texto_ver"
    android:layout_width="64dp"
    android:layout_height="64dp" />
```

- Para alterar a cor devemos utilizar app:tint

ANDROID - IMAGEVIEW

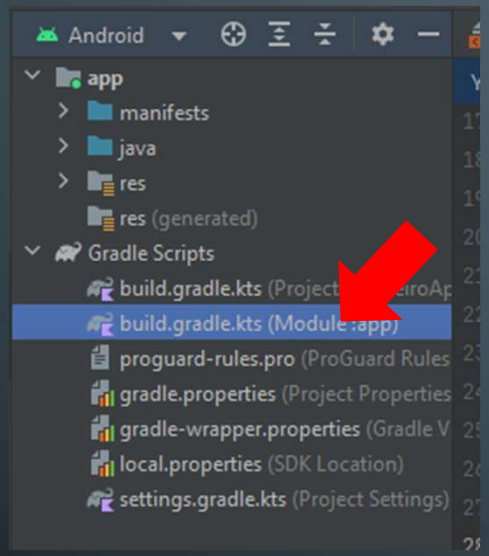
```
<ImageView
    android:src="@drawable/baseline_edit_24"
    android:layout_gravity="center_horizontal"
    app:tint="@color/texto_ver"
    android:layout_width="64dp"
    android:layout_height="64dp" />


<EditText
    android:id="@+id/edit_nome"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Nome" />
```



ANDROID - VIEWBINDING

- ViewBinding é o nome dado ao processo de associar a parte visual (o Layout) com o nosso código (programação).
- Antigamente era utilizado o findViewById que apesar de ainda funcionar, só deve ser utilizado ao dar manutenção em código legado.
- Primeira coisa que precisamos fazer é habilitar o viewBinding, ou seja, falar para o Android que esse aplicativo utiliza essa funcionalidade.
- Para habilitar precisamos editar o arquivo buildgradle.kts que esta dentro da pasta Grade Scripts.

ANDROID - VIEWBINDING



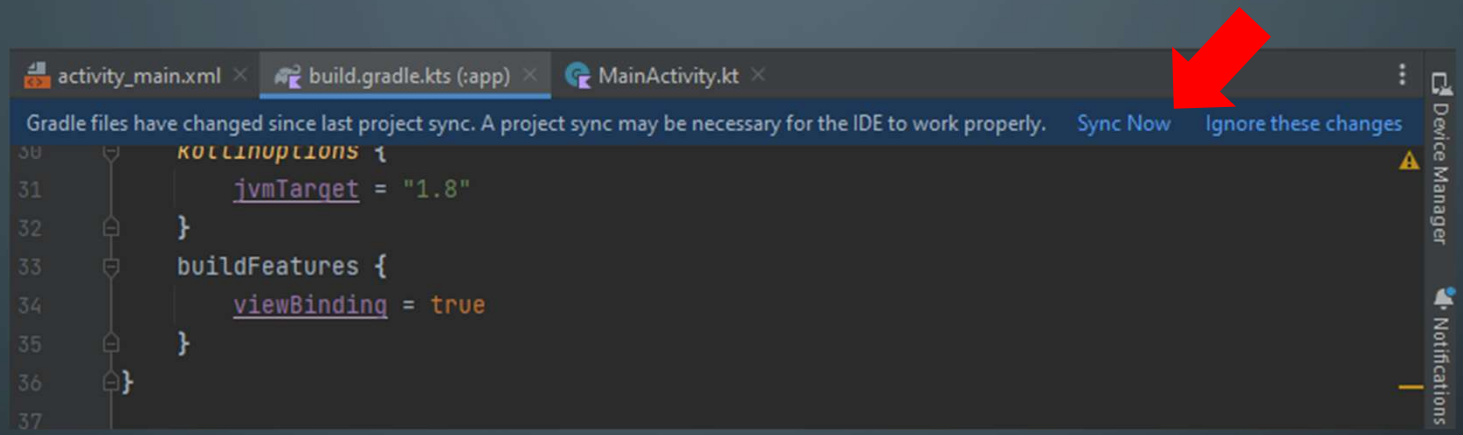
É o segundo arquivo, o que tem (Module:app) na frente

```
android {
    namespace = "com.tadseinstein.primeiroapp"
    compileSdk = 33

    defaultConfig {...}

    buildTypes {...}
    compileOptions {...}
    kotlinOptions {
        jvmTarget = "1.8"
    }
    buildFeatures {
        viewBinding = true
    }
}
```

Inclua o que esta dentro do quadrado vermelho, dentro da tag android.

ANDROID - VIEWBINDING

- Após a alteração, clique em Sync Now para validar as alterações:



- Pronto, seu projeto já consegue fazer o viewbinding entre as telas e o código fonte.

ANDROID - VIEWBINDING

- Para podermos utilizar os componentes visuais dentro do código precisamos instanciar uma variável para fazer essa ponte (viewbind). No código abaixo estamos instanciando a variável binding que conterá os componentes:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        var binding: ActivityMainBinding = ActivityMainBinding.inflate(layoutInflater)
```

- Devemos instanciar uma ActivityMainBinding, utilizando o método inflate com a variável layoutInflater.

ANDROID - VIEWBINDING

- Agora a variável binding contem os componentes que foram criados na nossa tela. Apenas os componentes que colocamos um id.

```
var binding: ActivityMainBinding = ActivityMainBinding.inflate(layoutInflater)

binding.
}
    buttonGrava                                  Button
    editCelular                                  EditText
    editNome                                     EditText
    root (from getRoot())                        LinearLayout
    equals(other: Any?)                          Boolean
    hashCode()                                   Int
    toString()                                   String
```

- Note que no .xml utilizamos o nome do componente como button_grava e aqui aparece como buttonGrava, ele remove o _ e muda a próxima letra para maiúscula.

ANDROID - VIEWBINDING

- O problema é que a variável binding só existe dentro do método onde ela foi criada, de qualquer outro local ela não será acessível.
- Para resolver isso podemos ir instanciando ela em todos os métodos que utilizarão algum componente, mas isso não é viável pois vamos ficar gerando cópias dela na memória do aparelho.
- Podemos declarar ela no escopo da classe, fazendo com que ela seja acessível de toda a classe.

ANDROID - VIEWBINDING

```
class MainActivity : AppCompatActivity() {

    private var binding: ActivityMainBinding = ActivityMainBinding.inflate(layoutInflater)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        binding.
    }
}
```

```
buttonGrava                    Button
editCelular                    EditText
editNome                       EditText
root (from getRoot())          LinearLayout
```

- Mas assim temos outro problema, essa variável instancia o ActivityMainBinding, mas ele só é criado quando o onCreate é executado, portanto naquele local ele ainda não existe.

ANDROID - VIEWBINDING

- Para resolver isso, devemos deixar a criação da variável ali, mas fazer sua instancia apenas dentro do método onCreate.
- Para isso utilizamos um lateinit.

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        binding = ActivityMainBinding.inflate(layoutInflater)
```

ANDROID - VIEWBINDING

- Ainda não esta pronto...
- Ao fazer o binding o layout passa a existir dentro do nosso código, e para ter os acessos de forma correta precisamos colocar o raiz da nossa tela no ContentView de nosso fonte:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(binding.root)

    binding = ActivityMainBinding.inflate(layoutInflater)


}
```

ANDROID - VIEWBINDING

- Mas o binding ainda não foi instanciado e dessa forma nosso aplicativo dará um erro ao ser executado.
- Basta mudar a ordem para ficar tudo certo:

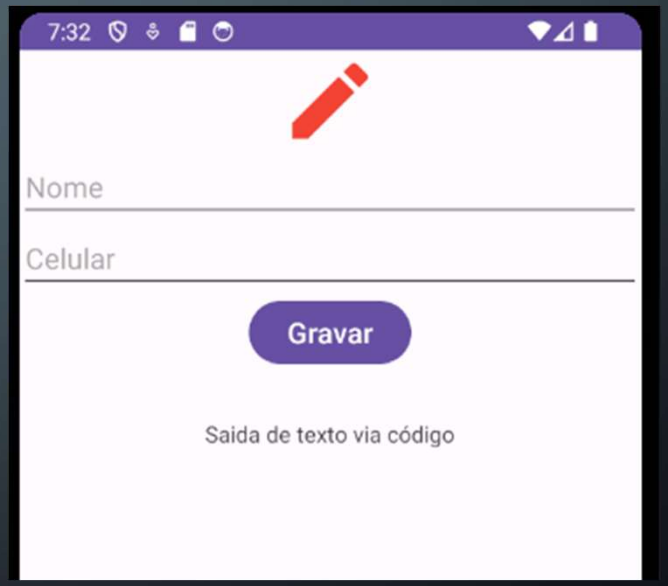
```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

    }
}
```

ANDROID - VIEWBINDING

- Para testar, vamos mudar o texto de um TextView via código:

```
<TextView
    android:id="@+id/text_resposta"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:layout_marginTop="30dp"
    android:text="Teste" />
```

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.textResposta.text = "Saida de texto via código"
    }
}
```



ANDROID – EVENTO DE CLICK NOS BOTÕES

- Vamos tratar os eventos de click nos botões.
- Utilizaremos o binding junto com o evento setOnClickListener. Este evento recebe uma interface, para isso teremos que implementá-la incluindo a mesma na declaração de nossa classe:

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
```

- Note que ele da erro, pois precisamos importar a View, basta colocar o mouse sobre a palavra em vermelho e digitar Alt+Shift+Enter:

```
class MainActivity : AppCompatActivity(), View.OnClickListener {

    private lateinit var binding: ActivityMa
```

Unresolved reference: View

Import class 'View' Alt+Shift+Enter More actions... Alt+Enter

ANDROID – EVENTO DE CLICK NOS BOTÕES

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
    privat
    overri
```

Class 'MainActivity' is not abstract and does not implement abstract member
public abstract fun onClick(p0: View!): Unit *defined in* android.view.View.OnClickListener

Implement members Alt+Shift+Enter More actions... Alt+Enter

- Agora deu um novo erro, pois a interface OnClickListener tem um método que somos obrigados a declarar (abstrato), vamos criar esse método:

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {...}

    override fun onClick(view: View) {

    }
}
```

ANDROID – EVENTO DE CLICK NOS BOTÕES

- Agora basta acessar nosso botão através do binding que fizemos e chamar o método setOnClickListener, assim toda vez que o botão for clicado ele chama o método onClick:

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.buttonGrava.setOnClickListener(this)

    }

    override fun onClick(view: View) {

    }
}
```

ANDROID – EVENTO DE CLICK NOS BOTÕES

- Quando nossa tela possuir mais de um botão, todos eles vão cair no mesmo método onClick, para saber qual botão foi clicado precisamos testar a propriedade id dele:

```
override fun onClick(view: View) {
    if( view.id == R.id.button_grava) {
        binding.textResposta.text = "O botão foi clicado"
    }
}
```

- No exemplo acima estamos apenas trocando o texto exibido pelo ViewText que temos na tela.

ANDROID – EVENTO DE CLICK NOS BOTÕES

- O código a ser executado quando um botão for clicado pode ficar grande (muitas linhas) e quando temos vários botões na tela o método onClick tente a ficar grande e confuso.
- Uma boa prática é criar outros métodos para colocar o código de cada botão, e utilizar o onClick apenas para chamar esses métodos:

```
override fun onClick(view: View) {
    if( view.id == R.id.button_grava) {
        btGrava()
    }
}

fun btGrava(){
    binding.textResposta.text = "O botão foi clicado"
}
```

ANDROID – TOAST

- Quando queremos apenas enviar um aviso rápido na tela, podemos utilizar o método Toast:

```
fun btGrava(){
    Toast.makeText( context: this,  text: "O botão foi clicado!", Toast.LENGTH_SHORT).show()
}
```

- O primeiro parâmetro se refere ao contexto, ou seja a tela atual;
- O segundo parâmetro é o texto que queremos exibir;
- O terceiro parâmetro é a duração da mensagem, podemos utilizar:
  - Toast.LENGTH_SHORT: exibição curta
  - Toast.LENGTH_LONG: exibição mais demorada.
- O show() no final da linha faz a mensagem ser exibida.

ANDROID – RECEBER VALORES DO EDITTEXT

- Para pegarmos o valor digitado em um EditText devemos pegar o atributo text e converter o mesmo para string com o toString():

```
fun btGrava(){
    val digitado = binding.editNome.text.toString()

    Toast.makeText( context: this, digitado, Toast.LENGTH_SHORT).show()
}
```

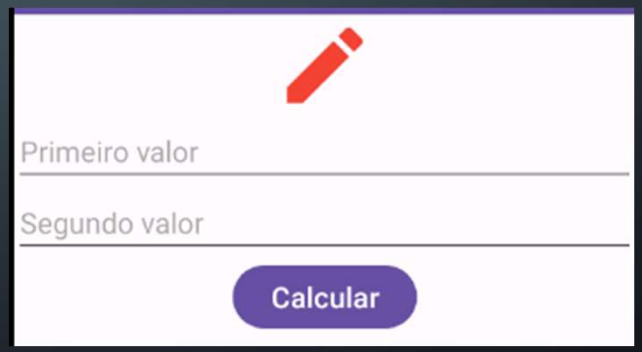
- Estamos colocando o texto digitado na variável digitado e exibindo a mesma na tela utilizando Toast.

ANDROID – RECEBER VALORES DO EDITTEXT

- Se quisermos converte o valor digitado em numero, devemos utilizar as funções de conversão como a toInt() ou toFloat():

```
<EditText
    android:id="@+id/edit_valor1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="number"
    android:hint="Primeiro valor" />

<EditText
    android:id="@+id/edit_valor2"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="number"
    android:hint="Segundo valor" />

<Button
    android:id="@+id/button_grava"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:text="Calcular"
    android:textSize="18sp" />
```

```
fun btGrava(){
    val valor1 = binding.editValor1.text.toString().toInt()
    val valor2 = binding.editValor2.text.toString().toInt()

    val soma = valor1 + valor2

    binding.textResposta.text = "A soma é $soma"
}
```



EXERCÍCIO

- Fazer um Aplicativo que calcule o total gasto com combustível, considerando a autonomia do veículo (KM por litro), distância percorrida e valor pago por litro.
- Calculo: Distância percorrida dividido pela autonomia multiplicado pelo valor do litro.
- Utilizar viewBinding
- Sugestão do visual do aplicativo: